RAG System

AI Ethics & Policy Q&A Assistant

Minmin Xiang

What is Retrieval-Augmented Generation (RAG)?

RAG enhances LLM responses by retrieving relevant documents from an external knowledge base before generation. This grounds answers in real source material rather than relying solely on model weights — reducing hallucination and increasing factual precision.

Why RAG for AI Policy Documents?

Factual grounding

LLMs hallucinate. RAG constrains answers to actual EU AI Act, OECD and UNESCO text.

Up-to-date knowledge

Policy documents update without retraining — just re-embed and reload ChromaDB.

Transparency

Retrieved chunks can be shown to users, making the system auditable.

Domain specificity

General LLMs lack deep regulatory knowledge. RAG fills that gap cheaply.

Step 1 — Data Preparation

Three authoritative AI policy PDFs form the knowledge base. They were selected for complementary scope: legal obligations (EU), normative principles (OECD), and ethical values (UNESCO).

EU AI Act

European Parliament, 2024

Legally binding framework classifying AI systems by risk level. Defines prohibited practices, obligations for high-risk systems, and conformity assessment procedures.

OECD AI Principles

OECD, 2019 (rev. 2024)

Government-endorsed principles for trustworthy AI covering transparency, accountability, robustness, and inclusive growth.

UNESCO Recommendation

UNESCO, 2021

Global ethical framework emphasising human rights, environmental sustainability, and inclusive development in AI deployment.

Chunking Strategy

Each PDF is loaded with PyPDFLoader, then split into overlapping chunks.
chunk_overlap=200 preserves sentence context across chunk boundaries, preventing relevant sentences from being cut off between adjacent chunks.

```
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
```

Step 2 — Embedding Generation

Each text chunk is converted into a dense vector using a Sentence Transformer model. Two models are evaluated to compare retrieval quality and inference speed.

`all-MiniLM-L6-v2`

Dimensions:	384 dimensions
Speed:	~0.05 s / query
Quality:	Good general-purpose retrieval
Memory:	~80 MB

`all-mpnet-base-v2`

Dimensions:	768 dimensions
Speed:	~0.09 s / query
Quality:	Superior on dense legal / policy language
Memory:	~420 MB

Embedding function — same interface for both models:

```
def embed(model, text):
    return model.encode([text], normalize_embeddings=True).tolist()[0]
```

Step 3 — Vector Database Setup with ChromaDB

ChromaDB is a local, persistent vector database. Two independent collections are created — one per embedding model — so retrieval quality can be compared fairly on identical queries.

ChromaDB Architecture

1. **PersistentClient**

   Stores data on disk at ./rag_db — survives notebook restarts

2. **col_minilm**

   All chunks embedded with MiniLM (384-dim vectors)

3. **col_mpnet**

   All chunks embedded with MPNet (768-dim vectors)

4. **Query flow**

   Embed query → cosine nearest-neighbour → return top-k chunks

Collection initialisation:

```
client = chromadb.PersistentClient(path="./rag_db")

def fresh_collection(name):
    try:
        client.delete_collection(name)
    except Exception:
        pass
    return client.create_collection(name)

col_minilm = fresh_collection("col_minilm")
col_mpnet  = fresh_collection("col_mpnet")

col_minilm.add(documents=text_lines, embeddings=embs1,
ids=ids)
col_mpnet.add( documents=text_lines, embeddings=embs2,
ids=ids)
```

Step 4 — Integration with Hugging Face LLMs

Two instruction-tuned seq2seq models are loaded locally. Both use the same wrapper function so they are fully interchangeable within the pipeline.

Model	Size	Avg Gen Time (CPU)	Answer Quality
`google/flan-t5-base`	~250 MB	**2.5 s**	Short, often incomplete
`google/flan-t5-large`	~800 MB	**153.5 s**	Detailed, contextually accurate

LLM loader — identical interface for both models:

```
def load_flan_t5(model_name):
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
    def generate(prompt, max_new_tokens=256):
        inputs = tokenizer(prompt, return_tensors="pt", truncation=True, max_length=512)
        with torch.no_grad():
            outputs = model.generate(**inputs, max_new_tokens=max_new_tokens)
        return [{"generated_text": tokenizer.decode(outputs[0], skip_special_tokens=True)}]
    return generate
```

Step 5 — Prompt Engineering: Three Templates

Prompt design significantly affects answer quality. Three framing strategies are tested to measure how instruction style influences LLM output quality and generation time.

Basic

Minimal framing. Can cause the model to go off-topic when context is ambiguous. Fastest (57.2 s avg).

```
PROMPT_BASIC = (
 "Answer the question using
only the context below.\n\n"
 "Context:
{context}\n\nQuestion:
{question}\nAnswer:"
)
```

XML Tags

Structured <context>/<question> tags. Cleaner parsing, especially for larger contexts. Moderate latency (74.0 s avg).

```
PROMPT_XML = (
 "Use the information enclosed
in <context> tags to provide
an answer to the "
 "question enclosed in
<question> tags.\n"
 "
<context>\n{context}\n</contex
t>\n"
 "
<question>\n{question}\n</ques
tion>"
)
```

Expert

Role-based — frames LLM as a policy expert. Most precise answers but longest generation (102.8 s avg).

```
PROMPT_EXPERT = (
 "You are an AI policy expert.
Using only the provided
context, give a precise "
 "and factual answer.\n\n"
 "Context:\n{context}\n\n"
 "Question: {question}\n\n"
 "Answer (based strictly on
the context above):"
)
```

Avg generation latency (flan-t5-large, CPU): Basic: 57.2 s XML Tags: 74.0 s Expert: 102.8 s — Expert writes longer, richer answers.

Step 6 — End-to-End RAG Pipeline

The rag() function ties every component together and measures retrieval and generation latency independently, enabling fine-grained benchmarking of all configurations.

```
def retrieve(question, collection, emb_model, k=3):
    q_emb = embed(emb_model, question)
    res = collection.query(query_embeddings=[q_emb],
n_results=k)
    return res["documents"][0]

def rag(question, collection, emb_model, llm, prompt_template,
k=3):
    t0 = time.time()
    docs = retrieve(question, collection, emb_model, k)
    t_ret = time.time() - t0

    context = "\n\n".join(docs)
    prompt = prompt_template.format(context=context,
question=question)

    t1 = time.time()
    answer = llm(prompt, max_new_tokens=256)[0]
["generated_text"]
    t_gen = time.time() - t1

    return {"answer": answer,
            "t_retrieve": round(t_ret, 3),
            "t_generate": round(t_gen, 3)}
```

Retrieval timing

Clocks embedding of query + ChromaDB nearest-neighbour search together.

Context assembly

Top-k chunks joined with double newlines — readable passage for the LLM.

Generation timing

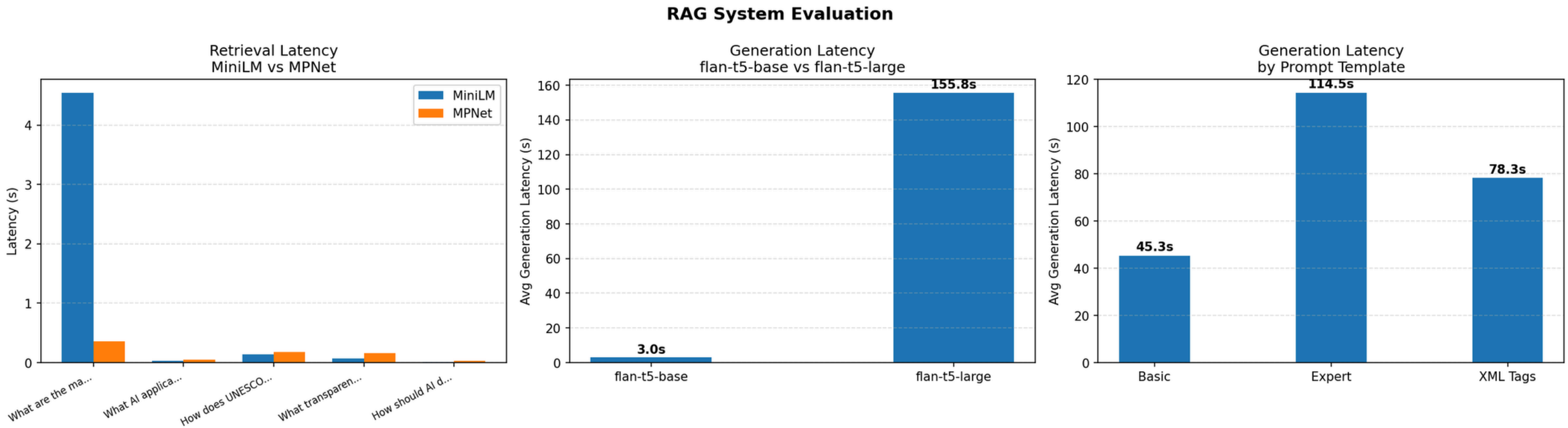
Clocks only the LLM forward pass, not tokenization or I/O overhead.

Structured output

Returns answer + both latencies — the evaluation loop logs every config.

Step 7 — Evaluation Results

The evaluation grid: 5 questions × 2 embedding models × 2 LLMs × 3 prompt templates. Charts show latency across retrieval, generation, and prompt dimensions.



1 MPNet returns more semantically relevant chunks for dense legal text, confirmed by both qualitative comparison and keyword accuracy scores (MPNet: 0.60 vs MiniLM: 0.53).

2 flan-t5-large is 61× slower than base (153.5 s vs 2.5 s). Quality gain is clear but GPU is essential.

3 Expert prompt adds ~80% latency over Basic (102.8 vs 57.2 s) because it generates longer answers.

Step 7 — Parameter Optimisation: Tuning k

After selecting the best embedding model and LLM, we tune k — the number of retrieved chunks. More context improves coverage but increases prompt length and generation time.

k (chunks)	Retrieval Time	Generation Time	Answer Quality	Verdict
k = 1	~0.07 s	fastest	One-line answers. Misses important context.	**Too sparse**
k = 3	~0.09 s	moderate	Complete, well-grounded answers consistently.	**Optimal**
k = 5	~0.11 s	27% slower than k=3	Marginal quality gain; adds noise and longer prompts.	Diminishing returns

k tuning experiment (MPNet + flan-t5-large + Expert prompt):

```
test_q = "What transparency obligations apply to AI systems under the EU AI Act?"

for k in [1, 3, 5]:
    r = rag(test_q, col_mpnet, emb_model_2, llm_large, PROMPT_EXPERT, k=k)
    print(f"k={k} | t_ret={r['t_retrieve']}s | t_gen={r['t_generate']}s | {r['answer']}")
```

Step 8 — Gradio Conversational Interface

A Gradio web interface exposes the full pipeline to end users. All parameters (embedding model, LLM, prompt style) are configurable at runtime without restarting the notebook.

Model & Prompt Selection

Dropdowns to switch embedding model (MiniLM/MPNet), LLM (base/large), and prompt style at runtime.

Live Latency Display

Separate text boxes show retrieval and generation latency after each query.

Context Viewer

Accordion shows exact top-3 chunks from ChromaDB — system is fully transparent.

Conversation History

Full chat history via gr.Chatbot renders a WhatsApp-style multi-turn dialogue.

Example Queries

Clickable examples let new users explore the system immediately.

Prompt Preview

Collapsible accordion shows the active prompt template for educational purposes.

Core respond() callback:

```
def respond(message, history, emb_choice, llm_choice, prompt_choice):
    col, emb = EMB_MAP[emb_choice]; llm = LLMS[llm_choice]; pt = PROMPTS[prompt_choice]
    r = rag(message, col, emb, llm, pt); history.append((message, r["answer"]))
```

Qualitative Analysis — What Differences Look Like

Beyond latency numbers, we compared what each configuration actually retrieves and generates for the same question. The differences are significant and directly affect usefulness.

Retrieval: MiniLM vs MPNet — query: "What are the prohibited AI practices in the EU AI Act?"

MiniLM top result

Retrieves an OECD Principles passage about rule of law and human rights — a completely different document. 384-dim vectors conflate general AI ethics language across sources and fail to associate "prohibited practices" with EU Act text.

MPNet top result

Retrieves an EU AI Act passage about data subjects' rights and automated decision-making. Closer to the right document than MiniLM, but still misses Article 5 directly. 768-dim vectors correctly identify the source document but need better chunk coverage to reach the prohibition list.

Both models miss Article 5 here. One likely reason is that the prohibition list spans multiple pages, so it gets split across chunks — no single chunk contains the full list. Increasing chunk size or adding a re-ranking step could help with this.

Answer quality comparison ("Which AI systems are considered high-risk under the EU AI Act?", MPNet + flan-t5-large, k=3):

Basic prompt

"harmful bias, human rights including safety, security, and privacy, as well as labour rights..."

Expert prompt

"AI systems identified as potential risks to human rights should be broadly tested before deployment..."

Both prompts draw from UNESCO context rather than EU Act Annex III — the retrieved chunks did not contain the specific high-risk category list. This shows that answer quality depends on retrieval quality first; prompt engineering alone cannot compensate for missing context.

Future Work

Future Improvements

→ Better LLM

Replace flan-t5 with Mistral-7B-Instruct (4-bit via bitsandbytes). Estimated: <1 s on GPU.

→ Re-ranking

Add cross-encoder re-ranker (ms-marco-MiniLM) after retrieval to boost chunk relevance.

→ Answer quality metric

Implement ROUGE-L or BERTScore against reference answers for automated accuracy tracking.

→ Multi-doc reasoning

Enable queries requiring synthesis across all three policy documents simultaneously.

Thank you!